

Claude Code 最佳实践全攻略：Karpathy Skills + claude-mem + Best Practice 三件套，让你的 AI 编程搭档

柒星栈 2026-04-15 👁 110 ⌚ 阅读18分钟

[关注](#)

导语

打开 Claude Code，信心满满地说“帮我重构这个模块”，然后眼睁睁看着它：加了三个你从没听过的依赖，搞出一堆你根本没要求的抽象层，还顺手把你的命名规范改了个遍。最后你花了比手写还多的时间来收拾残局。

我也是，直到我发现了 GitHub 上三个加起来超过 10 万 Star 的开源项目，这彻底改变了我和 Claude Code 的协作方式。

这篇文章，就是把我踩过的坑、试过的配置、验证过的效果，一次性给你讲清楚。

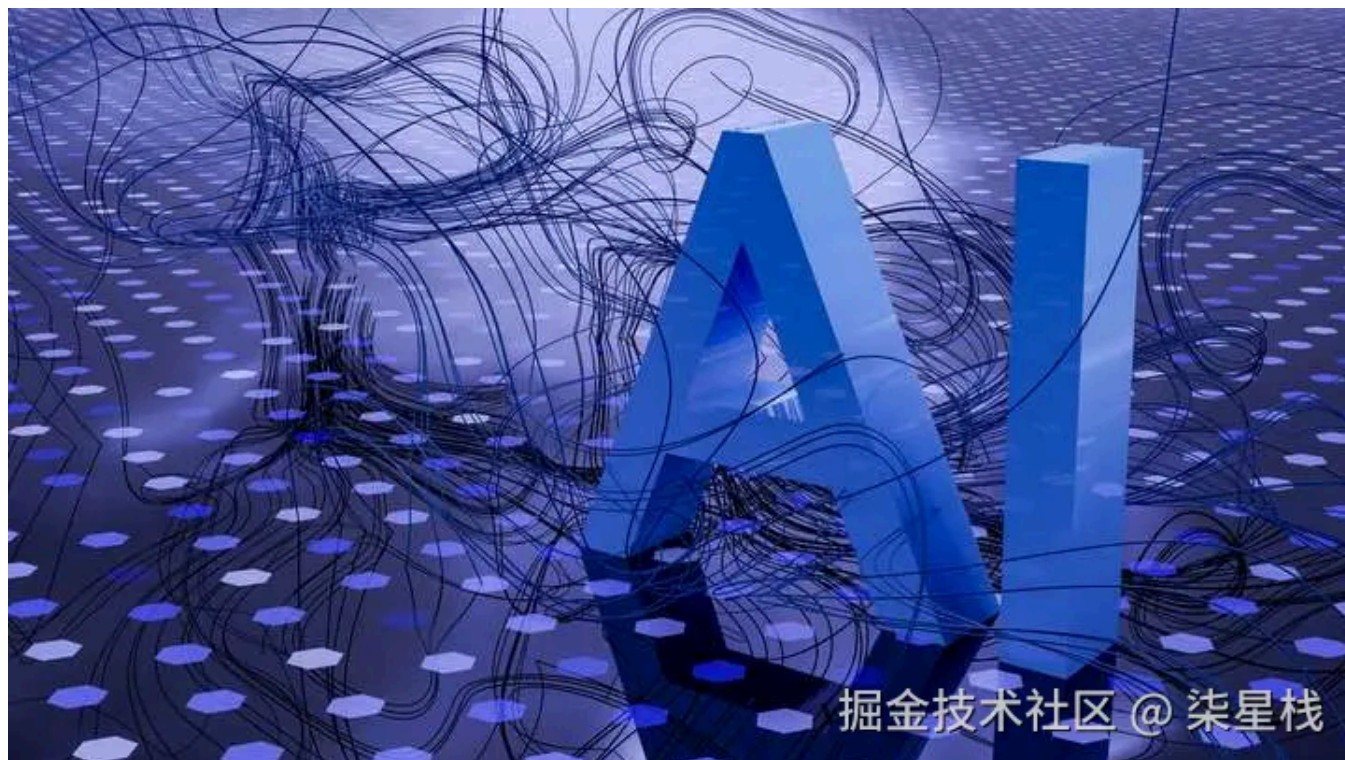
一个推文所引发的“配置革命”

这件事要从2026年初说起。Andrej Karpathy——也就是那个OpenAI联合创始人、前Tesla AI负责人、“vibe coding”这个词的发明者——在X上发布了一条推文。

他并没有分享什么新的工具，也没有推荐什么新的框架。他只是冷静地把LLM写代码时反复出现的几个毛病给列了出来：

- **过度工程**——你让它去修个按钮，它反倒给你搭了一整个组件库。
- **无视现有模式**——项目里明明用的是Zustand，它偏要引入Redux
- **乱加依赖**——你从没要求过的 npm 包，它悄悄就给你装上了

- **自信地犯错**——在不确定的时候不去询问，直接进行猜测，在猜错了之后还表现得特别笃定。



当LLM的错误变得可以被预测的时候，它们也就变得可以被预防了——Karpathy的这个洞察，简单却极具冲击力。

Karpathy 讲了一句特别扎心的话：“如果这些错误是可预测的，那它们就是可预防的。”

开发者 Forrest Chang 在看到这条推文之后，做了一件看起来简单但影响却十分巨大的事——他把 Karpathy 的观察提炼成了一个 CLAUDE.md 文件，并且将其放置在了 GitHub 上面。这也就是后来狂揽 13.7K Star 的 andrej-karpathy-skills。

与此同时，另外两个项目也在呈现爆发式增长的态势：thetodtmack 的 claude-mem (46.1K🌟) 解决了 Claude Code 的“失忆症”问题，shanraishan 的 claude-code-best-practice (42.3K🌟) 则把 Skills、Hooks、Subagents 的配置体系整理成了可供生产使用的级别的模板。

三件套各自拥有明确的分工，将它们组合起来就可以形成一套完整的 Claude Code 效率体系。

Karpathy 四原则：给 AI 装上“刹车”

先来说一说最核心的部分。andrei-karpathy-skills 这个项目，从本质上来讲就做了一件事：借助四条规则，把 Claude Code 从“自信的初级程序员”调教成“谨慎的高级工程师”。

原则一：Think Before Coding（先想清楚再动手）

这条规则要求 Claude 在编写任何代码之前，必须先用自然语言描述清楚它打算做什么，以及为什么要这么做。

说实话，这条规则解决的是 LLM 最让人头疼的问题——也就是“自信地犯蠢”。你有没有遇到过这种情况：Claude 直接就开始写代码，写完之后你一看，方向完全错了。它不是能力不行，是没想清楚就动手去做了。

加上这条规则之后，Claude 会先去输出一段思考过程：

▼ markdown  体验AI代码助手  代码解读 复制代码

```
1 我打算这样修改：
2 1. 在 UserService 中新增 validateEmail 方法
3 2. 复用现有的正则工具函数 utils/regex.ts
4 3. 不引入新依赖，使用项目已有的 validator 库
5
6 原因：现有代码中已有类似的验证逻辑，应保持一致性
```

我们可以这样来理解它的运行逻辑：它先把相关内容“说出来”，你再决定要不要让它“做出来”。要是方向出现了偏差，你只需要一句话就可以把它纠正过来，这样做比去修改代码要快得多。

原则二：Simplicity First（简单优先）

这条直接针对“过度工程”的毛病。规则很明确：

- 能用 3 行代码解决的，别写 30 行
- 能用现有工具的，别引入新依赖
- 能用简单实现的，别搞设计模式

我自己的体感是，加上这条规则之后，Claude 生成的代码量平均减少了约 40%。而且——这很关键——功能并没有发生变化。之前它动不动就给你搞出 Factory 模式、Strategy 模式，现在老老实实地写 if-else，反而更加清晰。

原则三：Surgical Changes（精准修改）

这条规则要求Claude只去修改需要修改的地方，不要顺手去“优化”周围的代码。

这样的痛点确实很常见。你让 Claude 去修复一个 bug，它还顺手把旁边三行被认为不够优雅的代码也进行了重构。而那三行代码牵扯到另外两个模块，一测试就全部出问题了。精准修改的核心要点就是：改一行是一行，不要去触碰不该碰的部分。

原则四：Goal-Driven Execution（目标驱动）

这条要求让 Claude 在执行过程当中持续对齐目标，要是发现偏离了原始意图，就必须停下来进行确认。

实际效果就是——你让Claude“给用户列表加个搜索框”，它不会在开展这项工作的过程当中，突然觉得“这个列表分页也该优化一下”，进而自作主张地进行修改。它会专注于搜索框的相关工作，完成之后再向你询问后续的安排。

安装方式

```
bash

1 # 克隆到你的 Claude Code skills 目录
2 git clone https://github.com/forrestchang/andrej-karpathy-skills.git
```

可以直接把项目当中的CLAUDE.md文件复制到你项目的根目录就可以。就这么简单——一个Markdown文件，以及四条规则，效果可以立竿见影。

不过说实话，这个项目也有它的局限所在。有人提出，Karpathy 的规则是写给独自编写代码的工程师的。要是你是 PM、或者带队做 SaaS 的创始人，你最昂贵的失败模式或许不是“过度工程”，而是“做错了东西”——也就是没人定义清楚用户痛点就开始编写代码。这个缺口，需要你自己去填补。

claude-mem：让 Claude 拥有“长期记忆”

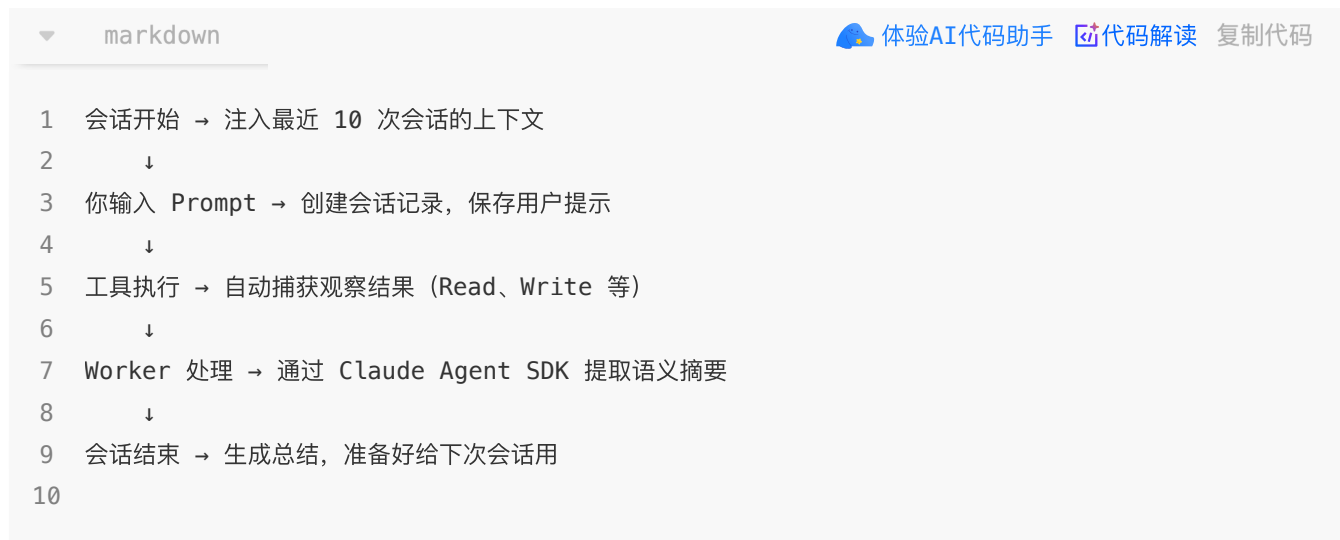
Karpathy 四原则解决了“行为规范”的问题，但还存在一个较大的问题——也就是 Claude Code 每次开启新会话的时候，记忆就会出现清零的情况。

你昨天花费了两小时的时间去教它项目的架构、命名规范以及数据库设计等内容.....今天再打开的时候，它就全都忘了。这样又得从头再来一遍。 claude-mem 就是用来填补这个坑的。46.1K

Star 并不是白拿的。

它是怎么工作的

claude-mem 的架构其实挺精巧的，它的核心流程是这样的：



关键在于——它并不是把原始对话全部存储下来（那会出现token爆炸的情况），而是运用语义压缩的方式。每次工具调用之后，claude-mem会自动捕获输入以及输出内容，然后借助Claude Agent SDK把它压缩成结构化的观察记录，存储到本地SQLite数据库当中。

下次开启新会话的时候，它会从数据库当中检索相关的上下文，只把你当前任务所需要的那部分注入进来。根据DataCamp的测试数据，这种方式比手动管理上下文每次会话可以节省约2250个token。

三层检索机制

claude-mem 运用的是分层检索的方式，并不是把所有内容一股脑全塞给你。：

1. 会话摘要层 — 最近 10 次会话的概要，自动注入
2. 语义搜索层 — 通过 MCP 工具按需检索，用 `claude-mem://` URI 引用具体观察
3. 文件夹上下文层 — 在项目目录自动生成 CLAUDE.md，包含活动时间线



记忆并不是简单地去堆数据，而是要做到精准的检索——claude-mem 的三层架构，就好比给 AI 装上了一个会“思考”的海马体。

安装方式

```
bash

1 # 方式一：npx 一键安装（推荐）
2 npx claude-mem install
3
4 # 方式二：从 Claude Code 插件市场安装
5 # 在 Claude Code 会话中执行：
6 /plugin marketplace add thedotmack/claude-mem
7 /plugin install claude-mem
```

完成安装后重启Claude Code就可以生效了。第一次使用的时候，claude-mem会比较“饥渴”——它需要学习你的项目，会记录大量新的观察内容。但经过几轮会话之后，它就进入稳定状态了，只在需要的时候进行检索和注入。

还有个挺贴心的功能，也就是多机同步。claude-mem-sync 可以借助 SSH 在不同机器之间同步观察记录以及会话摘要。这类细节也能说明，作者确实是在实战当中使用了这个工具。

隐私控制

claude-mem 支持双标签隐私系统，你可以把某些内容标记为私密，这类内容不会被注入到未来会话当中。处理 .env 文件、密钥之类的场景，这个功能是十分必要的。

Best Practice 仓库：生产级配置模板

要是把 Karpathy Skills 看作是“行为校准器”，claude-mem 是“记忆系统”，那 shanraishan 的 claude-code-best-practice 就是一套完整的配置蓝图。它拥有42.3K Star，是 Anthropic 黑客松获奖者的实战配置。

四层配置体系

这个仓库最值得关注的并非某个具体的配置，而是它所建立起来的一套分层决策框架：

层级	机制	定位	例子
L1	CLAUDE.md	项目记忆（建议性）	技术栈、命名规范、目录结构
L2	Skills	可复用的工作流	代码审计、品牌文案检查
L3	Hooks	强制执行的规则	禁止修改 .env、自动格式化
L4	Subagents	分工协作	代码审查员、迁移专员

这个分层实在是太重要了。不少人把各类内容都往 CLAUDE.md 里塞，结果导致文件变得越来越长，Claude 在每次会话的时候都要读取一遍，白白浪费掉 token。

正确做法是：把CLAUDE.md只用来放置最核心的项目信息，且将其控制在150条指令以内，详细流程放到Skills当中，安全规则放到Hooks里，复杂任务则交给Subagents来处理。

Hooks：不可逾越的红线

有个认知十分关键——Hooks 是强制性的，CLAUDE.md 则是建议性的。

这是什么情况呢？在 CLAUDE.md 当中写了“不要修改 .env 文件”，Claude 大概率会遵守这一要求，但并不是百分之百会做到。在 Hooks 里面配置一条 PreToolUse 规则，直接在系统层面去拦截对 .env 文件的读写操作，这样 Claude 就连文件内容都看不到了，这比“告诉它别碰”的安全程度要高很多。

▼ json  体验AI代码助手  代码解读 复制代码

```
1 // .claude/settings.json - 阻止 Claude 修改敏感文件
2 {
3   "hooks": {
4     "PreToolUse": [
5       {
6         "matcher": "Write|Edit",
7         "hooks": [
8           {
9             "type": "command",
10            "command": "echo 'Blocked: cannot modify .env files' && exit 1"
11          }
12        ]
13      }
14    ]
15  }
16 }
```

除此之外，你还可以在 settings.json 当中把敏感文件设为 deny，这样 Claude 就完全看不见这些文件了，比借助 Hooks 去拦截的方式更干净。

Skills：你的可复用工作流

Skills 是 Claude Code 在2026年当中最重要的功能升级之一。它比旧的 Commands 要更强大，支持附加文件、自动触发以及 Subagent 执行。

一个 Skill 的典型结构：

▼ bash  体验AI代码助手  代码解读 复制代码

```
1 .claude/skills/
2   quick-audit/
3     SKILL.md           # 技能定义
4     references/        # 参考文档
5     scripts/           # 配套脚本
6     examples/          # 示例文件
```


该核心特性具备自动触发的能力——你可以在 SKILL.md 的 description 当中写明对应的触发条件，比如“当编写或者编辑内容文件时使用”。这样一来就不用手动进行调用，Claude 会依据上下文自动完成匹配以及执行的操作。

Subagents：让 Claude 团队作战

Subagents 是 Claude Code 的“分工”机制。每一个 Subagent 都拥有属于自己的上下文窗口、工具权限以及模型配置。

一个特别实用的模式是“测试时间计算”也就是test time compute——让一个Subagent去编写代码，另一个Subagent也就是用同一个模型来审查代码。因为它们各自拥有独立的上下文窗口，审查者不会被编写代码时的假设以及偏见所影响。

▼ yaml  体验AI代码助手  代码解读 复制代码

```
1 # .claude/agents/code-reviewer.md
2 ---
3 name: Code Reviewer
4 model: claude-sonnet-4-20250514
5 tools:
6   - Read
7   - Grep
8   - Glob
9 ---
10
11 你是一个严格的代码审查员。你的职责：
12  - 发现 bug 和安全漏洞
13  - 检查是否遵循项目规范
14  - 不修改代码，只给出审查意见
```



Subagents 的核心要义：并非由单一个体去完成所有的工作，而是让每一个 Agent 都专注于去做自己最擅长的那一个环节。

best-practice 仓库当中还有一个十分巧妙的例子，也就是天气 Agent。它属于一个专门用来获取天气数据的 Subagent，并且预加载了 Open-Meteo API 的调用方式。当主 Agent 需要天气数据的时候，就可以直接把任务委派给这个专业的 Agent，不需要在主上下文当中塞入 API 文档。

配置层级

Claude Code 的配置有清晰的层级关系：

▼ objectivec		 体验AI代码助手	 代码解读	复制代码
1	~/ .claude/CLAUDE.md	→ 全局个人设置（所有会话生效）		
2	项目根目录/CLAUDE.md	→ 项目级设置		
3	src/CLAUDE.md	→ 目录级设置（进入 src 目录时生效）		

这个渐进式披露的设计十分巧妙——全局配置得以保持精简，细节则按需进行加载。

上下文管理：被忽视的“隐形杀手”

平心而论，即便上述这三个项目表现得再出色，要是你不对上下文进行管理，那么整体的效果就会大打折扣。

Claude Code 的上下文窗口并不是无限的。每一次对话轮次，整个上下文都会被重新进行处理。当上下文变得越来越长时，Claude 的表现会出现明显下降——开始忘记之前的约定、重复犯下错误，甚至产生幻觉内容。

SmartScope 的分析文章当中有个十分醒目的观点：并非“坚持越久越好”，而是要以“会话会被污染”作为前提来开展相关操作。

三个关键操作

1. 勤用 `/clear`

官方最佳实践明确提到了要频繁使用 `/clear`。但具体到底多频繁呢？社区的经验是——当你感觉到 Claude 开始出现“跑偏”的情况时，果断进行清除。不要舍不得之前的上下文，如果已经安装了 claude-mem，那么重要的信息就已经被保存下来了。

2. 主动 `/compact`

自动压缩会在上下文占用达到阈值的时候触发，不过你也可以主动对其进行控制。关键的技巧在于，压缩的时候要把需要保留的内容告知给 Claude：

▼ bash

体验AI代码助手

代码解读

复制代码

1 /compact 保留 API 变更列表和已修改文件清单

你也可以在 CLAUDE.md 当中去添加一条常驻的指令：

▼

体验AI代码助手

代码解读

复制代码

1 压缩上下文时，始终保留：完整修改文件列表和当前测试状态

3. 按上下文单元拆分任务

不要在一个会话当中去完成所有的事情。正确的节奏应该是：

▼ arduino

体验AI代码助手

代码解读

复制代码

- 1 会话 1: 需求分析 + 方案设计 (Plan Mode)
- 2 ↓ /clear
- 3 会话 2: 核心功能实现
- 4 ↓ /clear
- 5 会话 3: 测试 + 清理

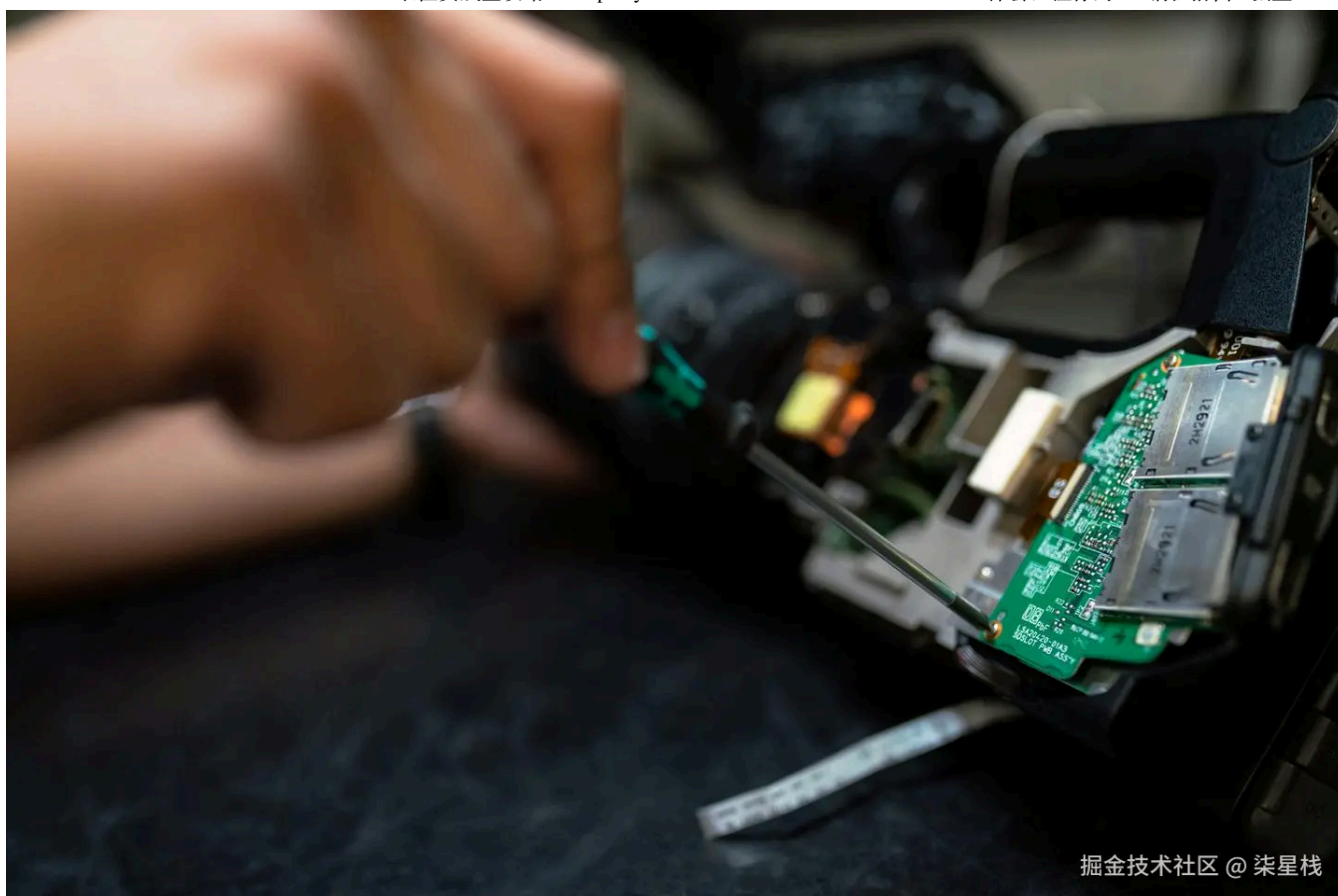
每个会话会从 CLAUDE.md（以及 claude-mem 的上下文注入）开始，带着上一步的关键结论，但不会带上一步的“噪音”，这样整体的质量和速度都能得到提升。



上下文就像是工作台一样，上面堆满了工具反而找不到需要使用的那一个。勤去做清理工作，才可以保持精准的状态。

三件套整合：我的实战配置方案

那么，该把这些内容拼接起来了。下面是我目前正在使用的配置方案，经过大概两个月的实战迭代，感觉它已经比较稳定了。



掘金技术社区 @ 柒星栈

不是去堆砌配置，而是去搭建体系——四层分别承担各自的职责，这样才能让每一个配置都发挥出最大的价值。

第一步：安装 Karpathy Skills

```
bash
1 cd your-project
2 # 把 Karpathy 的 CLAUDE.md 合并到你的项目配置中
3 curl -o CLAUDE.md https://raw.githubusercontent.com/forrestchang/andrej-karpathy-skills/
```

然后根据你的项目情况，在 CLAUDE.md 末尾追加项目专属配置：

```
markdown
1 # 项目专属配置
2
3 ## 技术栈
4 - 前端: React 19 + TypeScript 5.x + Tailwind CSS 4
5 - 状态管理: Zustand (不要用 Redux)
6 - 后端: Node.js 22 + Fastify
7
8 ## 命名规范
9 - 组件: PascalCase
```



```
10 - 工具函数: camelCase
11 - 常量: UPPER_SNAKE_CASE
12 - 文件: kebab-case
13
14 ## 关键约束
15 - 所有 API 响应使用 { success, data, error } 格式
16 - 数据库迁移放在 src/database/migrations/
17 - 不要引入新的 UI 库, 使用项目已有的 shadcn/ui
```

第二步：安装 claude-mem

```
1 npx claude-mem install
```

将 Claude Code 重启，并且去确认 Worker 服务的运行状态是否正常：

```
1 npm run worker:status
```

首次使用的时候，claude-mem会花费几轮会话来学习你的项目。不要着急，让它运行几天，你就会明显感受到新会话的“起步速度”变快了。

第三步：配置 Hooks 和 Subagents

可以从best-practice仓库挑选你所需要的配置：

```
1 git clone https://github.com/shanraishan/claude-code-best-practice.git
```

我建议至少把这几个给配置好：

必须的 Hook — 自动格式化：

```
1 {
2   "hooks": {
3     "PostToolUse": [
```

```
4      {
5          "matcher": "Write|Edit",
6          "hooks": [
7              {
8                  "type": "command",
9                  "command": "npx prettier --write $CLAUDE_FILE_PATH"
10             }
11         ]
12     }
13 ]
14 }
15 }
```

必须的 Hook — 敏感文件保护：

在 `settings.json` 的 `permissions` 中设置：

▼ json

 体验AI代码助手  代码解读 复制代码

```
1 {
2   "permissions": {
3     "deny": [".env", ".env.*", "*.pem", "*.key", "credentials/*"]
4   }
5 }
```

推荐的 Subagent — 代码审查员：

▼ yaml

 体验AI代码助手  代码解读 复制代码

```
1 # .claude/agents/code-reviewer.md
2 ---
3 name: Code Reviewer
4 model: claude-sonnet-4-20250514
5 tools:
6   - Read
7   - Grep
8   - Glob
9 ---
10
11 你是严格的代码审查员。检查：
12 1. 是否遵循项目 CLAUDE.md 中的规范
13 2. 是否存在 bug 或安全漏洞
14 3. 是否有不必要的依赖引入
15 4. 修改是否精准（没有多余的改动）
16
17 只给出审查意见，不修改代码。
```

第四步：建立日常节奏

最后，给你一个日常使用节奏的参考：

CSS

 体验AI代码助手

 代码解读

复制代码

1

每天开始 → 启动 Claude Code → claude-mem 自动注入上下文

2

↓

3

新功能 → Plan Mode 先规划 → /clear → 实现

4

↓

5

修 Bug → 精准描述问题 → 让 Claude 先思考再动手

6

↓

7

代码审查 → 委派给 code-reviewer Subagent

8

↓

9

感觉跑偏 → 果断 /clear → 从 CLAUDE.md 重新开始

10

↓

11

每天结束 → 关闭会话 → claude-mem 自动生成总结

效果对比：裸奔 vs 配装

我把同一个任务，也就是给现有项目添加用户搜索功能，拿来做了对比测试：

指标	裸奔 Claude Code	三件套配装
新增依赖	3 个 (lodash.debounce, react-select, fuse.js)	0 个 (复用项目已有工具)
代码行数	287 行	164 行 (-43%)
偏离原始需求的改动	5 处	0 处
需要人工修正的次数	4 次	1 次
第二次会话是否记得上下文	❌ 完全不记得	✅ 自动注入相关上下文

数据能够说明一切。同样的Claude，同样的模型，要是配置不同，效果的差距就会十分巨大。

一些诚实的补充

说了这么多好话，也得聊一聊相关的局限性。

Karpathy Skills 的四项原则对于个人开发者来说已经足够使用，但对于产品团队而言，它缺少了一个部分——也就是“做对的东西”。正如 masteringproducthq.com 的分析所指出的，至少有三分之一的 AI 辅助开发工作，其问题并非“代码写错了”，而是“从一开始就不该开发这个功能”。这个缺口，CLAUDE.md 无法解决，需要依靠你自身的产品判断力。

claude-mem 在项目初期会有“学习期”，前几轮会话所注入的上下文量比较大，token 消耗会偏高。另外，它的语义搜索是否会导致不同项目的上下文“串台”？从现有文档来看，它更像是“对过去会话的快速搜索”而非“后台大脑”，跨项目隔离需要你自行留意。

Best Practice 仓库 的配置十分全面，但全面的同时也代表着复杂。要是你是 Claude Code 的新手，不要一上来就把 30 个 Skills、9 个 Hooks、5 个 Subagents 全都装上。可以从 CLAUDE.md 加上一个 Hook 再加上一个 Subagent 开始，逐步进行添加。

写在最后

Claude Code 的顶尖水准，其实并不在模型本身。就拿同一个 Claude Sonnet 4 来说，未做任何额外配置和经过精调配置的情况之间，差距至少达到了三倍的效率。

Karpathy 的四原则用来解决“行为规范”相关的问题，claude-mem 用来解决“记忆持久化”相关的问题，best-practice 仓库用来解决“配置体系化”相关的问题。这三件套组合起来之后，Claude Code 才真正从“工具”转变成了“搭档”。

不过说到底，这些配置都只是手段，而并非目的。真正的目的是让你把时间花在真正需要人类判断的地方，而不是反复去纠正 AI 的低级错误。

要是你已经在使用 Claude Code，今天就可以动手操作起来：先安装 Karpathy Skills，再安装 claude-mem，最后按照实际需求从 best-practice 仓库当中挑选合适的配置。一个下午的时间就能够搭建好基础框架，使用效果当天就能真切感受到。

标签：

人工智能

话题：

每日精选文章

评论 0



[登录 / 注册](#) 即可发布评论!



暂无评论数据

目录

[收起 ^](#)

导语

一个推文所引发的“配置革命”

Karpthy 四原则：给 AI 装上“刹车”

原则一：Think Before Coding（先想清楚再动手）

原则二：Simplicity First（简单优先）

原则三：Surgical Changes（精准修改）

原则四：Goal-Driven Execution（目标驱动）

安装方式

claude-mem：让 Claude 拥有“长期记忆”

它是怎么工作的

三层检索机制

安装方式

隐私控制

Best Practice 仓库：生产级配置模板

四层配置体系

Hooks：不可逾越的红线

Skills：你的可复用 workflow

Subagents：让 Claude 团队作战

配置层级

上下文管理：被忽视的“隐形杀手”

三个关键操作

三件套整合：我的实战配置方案

第一步：安装 Karpathy Skills

第二步：安装 claude-mem

第三步：配置 Hooks 和 Subagents

第四步：建立日常节奏

效果对比：裸奔 vs 配装

一些诚实的补充

写在最后

精选内容

一天一个开源项目（第86篇）：VibeVoice —— 微软开源的前沿语音 AI，单次处理 90 分钟多说话人音频

冬奇Lab · 54阅读 · 0点赞

要裂开了！ChatGPT要手机号验证了？注册Codex要求验证电话号码怎么办？2026年登陆Codex要手机号验...

Mac的实验室 · 603阅读 · 1点赞

薅满火山引擎每天数百万免费 Tokens：我写了一个自动轮换代理

大橙子打游戏 · 91阅读 · 2点赞

MaxFrame 视频帧智能分析：从视频到语义向量的端到端分布式处理

阿里云大数据AI... · 29阅读 · 0点赞

为什么你的 Agent 读了文件，却好像什么都没读到？我扒了 Claude Code 源码，找到了原因

两万五千个小时 · 45阅读 · 2点赞

为你推荐

一天一个开源项目（第70篇）：claude-code-best-practice - 从 Vibe Coding 迈向 AI 原生开...

冬奇Lab | 19天前 | 147 点赞 评论 人工智能 开源 资讯

andrej-karpathy-skills：一篇搞懂 Karpathy 的 AI 编程四原则

递归不能太深 | 1天前 | 13 点赞 评论 AI编程

别再手把手教AI了-Skills聚合站与仓库全攻略

不想起名字1111111 | 13天前 | 126 点赞 评论 人工智能

别再裸用 Claude Code 了！32 个亲测Skills + 8 个 MCP，开发效率直接拉满！

蝎子菜菜爱打怪 | 1月前 | 47k 点赞 95 评论 后端 Claude Java

Claude Code 最佳实践仓库拆解：一篇看懂 Agentic Engineering 落地方法

小碗细面 | 1月前 | 490 点赞 4 评论 AI编程 Claude

不仅仅是代码助手：用 Plugins 将 Claude Code 打造成你的专属工具链 - 实战篇（2/4）

RobinMin | 6月前 | 383 点赞 1 评论 命令行 Medium

Claude Code Skills 资源大盘点：导航站 + GitHub 精选仓库全整理

爱吃的小肥羊 | 1月前 | 522 点赞 4 评论 1 评论 人工智能 AI编程

Claude Code 实战指南：从入门到精通，凯神带你忘本其他 AI

lenyan | 1月前 | 548 点赞 3 评论 1 评论 AI编程 Claude

本周值得关注的几个 AI 开源项目：Claude Code、Agent 和大模型工具

用户6182752318481 | 5天前 | 36 点赞 1 评论 评论 LLM GitHub

200+AI Agent Skills实测清单 | 安装教程+避坑指南+一步API集成方案

一步API | 1月前 | 430 点赞 1 评论 评论 后端

Claude Code 最佳实践导读：上下文是第一资源

贵慙_Derek | 1月前 | 97 点赞 评论 评论 人工智能

Claude Code 通识 - claude_0x01

用户22358621820 | 12天前 | 85 点赞 1 评论 评论 AI编程 Claude

Agno 开发教程（十一）：深入理解与实践 Skills（技能）

Cursor 独有的 12 个技巧：这些是 Claude Code 没有的

清汤饺子 | 1月前 | 👁 2.5k | 👍 29 | 💬 2

前端后端AI编程

Claude Code 实战：10 个让效率翻倍的技巧

littlex | 5月前 | 👁 1.6k | 👍 2 | 💬 评论

Claude

